

PRACTICAL-03

AIM: Implementation of max-heap sort algorithm

MAX-HEAP SORT

ALGORITHM:

Input : Array $A[0, \dots, n-1]$
Output : max Heap
step 1: start
step 2: Read n and array A
step 3: set $i = \frac{n}{2} - 1$
step 4: Repeat while $i > 0$
step 5: set $largest = i$
step 6: set $left = 2*i + 1$ and $right = 2*i + 2$
step 7: If $left < n$ and $A[left] > A[largest]$
then $largest = left$.
step 8: If $right < n$ and $A[right] > A[largest]$
then $largest = right$.
step 9: If $largest \neq i$, swap $A[i]$ and $A[largest]$
step 10: Apply maxHeapify ($A, n, largest$)
step 11: Decrement $i = i - 1$
step 12: Repeat until $i < 0$
step 13: Display the max heap
step 14: stop

PROGRAM:

```
#include <iostream>
using namespace std;
```

```
void MaxHeapify(int arr[], int size, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < size && arr[l] > arr[largest])
        largest = l;

    if (r < size && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        MaxHeapify(arr, size, largest);
    }
}

void HeapSort(int arr[], int size) {
    for (int i = size / 2 - 1; i >= 0; i--)
        MaxHeapify(arr, size, i);

    for (int i = size - 1; i >= 1; i--) {
        swap(arr[0], arr[i]);
        MaxHeapify(arr, i, 0);
    }
}

int main() {
    int size;
    cout << "Enter number of elements: ";
    cin >> size;

    int arr[size];
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    HeapSort(arr, size);

    cout << "Sorted array: ";
    for (int i = 0; i < size; i++) {
```

```

    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

OUTPUT:

```

Enter number of elements: 5
Enter 5 elements:
9
0
6
1
7
Sorted array: 0 1 6 7 9

```

TIME COMPLEXITY:

```

maxHeapify (arr[], size, i) → TC(n)
{
    int length = i; → 1
    int l = (2*i); → 1
    int r = (2*i)+1; → 1
    while (l < size && arr[l] > arr[i])
    {
        largest = l; → 2
    }
    while (r < size && arr[r] > arr[largest])
    {
        largest = r; → 1
    }
    if (largest != i) → 1
    {
        swap (arr[largest], arr[i]);
        maxHeapify (largest, size); → TC(n)
    }
}

HeapSort (i = size/2; i > 1; i--) → n/2
{
    maxHeapify (arr, size-i); → TC(n)
}

for (i = size; i > 1; i--) → n-1
{
    swap (arr[i], arr[1]); → 1
    maxHeapify (arr, i-1); → TC(n)
}
}

```


$$T(n) = T(n/2) + 1$$

using master method

$$f(n) = 1$$

$$a = 1 \quad b = 2 \quad k = 0 \quad p = 0$$

$$\log_b a = \log_2 1 = \frac{\log 1}{\log 2} = 0 = k$$

$$p > -1 \Rightarrow 0 > -1$$

$$T(n) = O(n^k \log^{p+1} n)$$

$$= O(n^0 \log^{0+1} n)$$

$$= O(\log n)$$

$$\text{max heapify} = O(\log n)$$

Time and cases

$$1) \text{ Best case : } O(n \log n)$$

$$2) \text{ Average case : } O(n \log n)$$

$$3) \text{ Worst case : } O(n \log n)$$

Build max heap

$$T(n) = n/2 \times \log n$$

$$= O(n \log n)$$

Heap sort

$$T(n) = (n-1) \times \log n$$

$$= O(n \log n)$$

Worst case

$$T(n) = T(n/2) + O(1)$$

$$= O(\log n)$$

for all $n-1$ times

$$T(n) = O(n) + (n-1) \times O(\log n)$$

$$= O(n \log n)$$

Average case

same time as worst case

Overall heap sort

$$T(n) = O(n \log n) + O(n \log n)$$

$$= O(n \log n) \rightarrow \text{best case}$$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.